Lesson 3: Take Your Shot

TIC-80

VIDEO GAME CREATION

This week: spawning objects at runtime. Your job is to make your player shoot a projectile that flies up the screen and cleans itself up when it leaves - the first piece of your capstone's actual shooting mechanic.

Open your own [YourName]Game project from Lesson 2 - don't start a new one. Everything this week builds on the player object you already have.

Tip: this lesson's Assets folder has a ready-to-use placeholder projectile image (bullet_yellow.png) and a new sound effect (sfx_shoot.wav) - you don't need to find or make your own art today.

Setup

1. Launch TIC-80 and load your cart from Lesson 2 (or export/re-import it if you saved to disk - see the Lesson 1 Install Guide for TIC-80's save/load commands).
2. Open the Sprite Editor (F2) and draw a small bullet shape on sprite index 2 (index 1 is still your player character from Lesson 2).

Level 1 - Minimal (MVP)

Pressing the fire key spawns one projectile that flies upward and disappears when it leaves the screen. This is a complete finish for today - reaching this is enough to move on to next week.

3. Press Esc to reach the code editor.
4. Add a bullets table near the top of your code (an empty Lua table that will hold one entry per active bullet).
5. Inside your existing TIC() function, add a btnp(4) check (button 4 is Z, TIC-80's default "A" button) that inserts a new {x=x, y=y} entry into the bullets table when pressed.
6. Add a loop that walks the bullets table backwards each frame, moving each bullet up and drawing it, removing any that have gone off the top of the screen.
7. Press Esc, type run, press Enter, then press Z to test - a bullet should spawn at your character and fly upward, then vanish at the top.

```
bullets={}

function TIC()
 cls(0)
 if btn(3) then x=x+1 end
 if btnp(4) then
```

```
  table.insert(bullets,{bx=x,by=y})
 end
 for i=#bullets,1,-1 do
  local b=bullets[i]
  b.by=b.by-2
  if b.by<0 then
   table.remove(bullets,i)
  else
   spr(2,b.bx,b.by,0)
  end
 end
 spr(1,x,y,0)
end
```

Level 2 - Complete (Alpha/Beta)

Firing repeatedly produces multiple independent projectiles on screen at once, each moving and disappearing correctly, and rapid-fire is capped so holding the key down doesn't flood the screen.

- Test firing several times quickly - each press should insert its own entry into the bullets table, all moving and being removed independently. Because the loop walks every entry in the table each frame, this already works with no extra code.
- Add a cooldown so holding Z doesn't fire every frame: btnp() already only returns true on the single frame a button is first pressed (not every frame held, unlike btn()), so Level 1 is already immune to this specific bug.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Sound: play a "pew" sound the moment a bullet is fired. Open the SFX Editor (F4) and design a short descending-pitch effect in an empty slot, then call sfx(slot_number) right next to the table.insert() call.
- Visual: draw bullet sprite index 2 as visually distinct from the player (different shape or color), or use spr()'s flip/rotate argument to give the bullet a spin as it travels.
- Structure: keep the bullet-spawning check (btnp(4) + table.insert) and the bullet-updating loop (movement, drawing, and cleanup) as the two clearly separate pieces shown above, rather than folding bullet logic into the same lines that check player movement.

If Something's Wrong

- Bullets fire every frame instead of once per press: double-check you used btnp(4) (press) and not btn(4) (held) for the fire check.
- Bullets never appear: confirm you drew something on sprite index 2 in the Sprite Editor, and that spr(2, b.bx, b.by, 0) is inside the for loop, not outside it.
- "attempt to index a nil value" error: this usually means the bullets table wasn't declared before TIC() runs - confirm bullets={} is at the very top of your code, outside any function.

Want More?

Student Resource List > TIC-80 section, or tic80.com